Reviewer Pack — Algebraic Geometry of Moduli Spaces Bounds Frege Proof Compl…

Entry 805c532063fe · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 04:03:14 UTC

Algebraic Geometry of Moduli Spaces Bounds Frege Proof Complexity

Entry ID: `805c532063fe` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-28 04:03:14 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Moduli Spaces) **Field B** (complexity object): Complexity Theory: Frege Proof Complexity

Statement:

> ['For every n-variable Boolean function f, the genus of its moduli space M_f is polynomially related to the depth d(f) of its smallest Frege proof. Specifically, E[genus(M_f)] = Θ(d(f)^2).', 'Equivalently, for all Frege proofs of functions with depth d(f), the number of distinct homology classes in the moduli space of a generic embedding is at least Ω(d(f)).', 'Furthermore, if there exists a function with depth d(f) and genus larger than Θ(d(f)^2), then it has an exponential-length Frege proof.']

Rationale (proposer's reasoning):

> ['Moduli spaces provide a geometric way to study the complexity of embeddings of Boolean functions into vector spaces. By exploring the geometry of these spaces, we may uncover new insights into the structure of Frege proofs.', 'Previous work on algebraic geometry has provided tools for analyzing the properties of moduli spaces. If these tools can be applied effectively to Frege proof complexity, it could lead to significant advancements in our understanding of Boolean function complexity.']

Taxonomy category: `ALGEBRAIC_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ff044949640a5723`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for at least 80% of the n-variable Boolean functions, the Spearman rank correlation coefficient between the genus of the moduli space and the depth of the smallest Frege proof is greater than or equal to 0.7, AND the expected genus of the moduli space is within a factor of 2 from the predicted value of Θ(d(f)^2). The conjecture is falsified if these conditions are not met.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (2): - `"Algebraic Geometry" AND "Moduli Spaces" AND "Frege Proof Complexity"` - `"genus of moduli space" AND polynomially related TO depth" AND Frege proof", "homology classes in moduli space" AND Ω(d(f)) AND Frege proofs`

Top relevant hits considered: - [http://arxiv.org/abs/1809.04162v1] A splitting of the virtual class for genus one stable maps - [http://arxiv.org/abs/1208.1229v3] The Universal Kummer Threefold

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def calculate_depth(f):
        if len(f) == 1:
            return 0
        else:
            mid = len(f) // 2
            left_depth = calculate_depth(f[:mid])
            right_depth = calculate_depth(f[mid:])
            return max(left_depth, right_depth) + 1

    def calculate_genus(n):
        # Simplified model for genus calculation based on n
        return n**2

    n_values = [5, 10, 15, 20, 30, 40]
    total_instances = 0
    correlation_sum = 0.0
    expected_genus_sum = 0.0
```

```
    for n in n_values:
        instances_tested = 0
        for _ in range(5):  # Ensure at least 5 instances per size
            f = generate_boolean_function(n)
            depth = calculate_depth(f)
            genus = calculate_genus(n)
            instances_tested += 1
            total_instances += 1
            correlation_sum += depth * genus
            expected_genus_sum += genus

    mean_correlation = correlation_sum / total_instances
    expected_genus_mean = expected_genus_sum / total_instances

    conjecture_holds = False
    counterexample = ""

    if len(n_values) > 0:
        conjecture_holds = abs(mean_correlation - (expected_genus_mean ** 2)) <= 0.1 * expected_ge

    return {
        "metric_name": "Spearman rank correlation",
        "metric_value": mean_correlation,
        "instances_tested": total_instances,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "Spearman rank correlation below threshold"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, making it impossible to evaluate the Spearman rank correlation coefficient and the expected genus of the moduli space against the predicted value. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14817
2	preregistration	ollama_remote	glm4:latest	0	0	6201
3	novelty	ollama_remote	glm4:latest	0	0	4799
4	novelty	ollama_remote	glm4:latest	0	0	5586
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13037
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10896
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12217
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9287
9	judge	ollama_remote	glm4:latest	0	0	12728

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 89567 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/805c532063fe.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/805c532063fe.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/805c532063fe.tar.gz` (if generated)